

中国海洋大学

系统开发工具基础

实 验 报 告

第 4 周

姓 名 张子航

学 号 25020007176

授课教师 周小伟

实验日期 2026 年 9 月 14 日

一、课上实验检查

实验内容

复用第 10 题完成的 greetlab 工程基础，在 q13 中建立符合工业标准的轻量级本地质量检查体系。在 `pyproject.toml` 中添加 `ruff`（代码风格与 Lint）及 `pytest`（单元测试）的最小合规配置；在 `tests/test_cli.py` 中补充正常姓名参数测试 `test_normal_name` 以及纯空白姓名非法退出测试 `test_blank_name_exits_with_code_2`，确保两个测试均具备明确断言；编写自动化流水线脚本 `check.sh`，按序自动触发 `ruff format --check`、`ruff check` 与 `pytest -v`；全量修复潜在违规代码，严禁全局忽略规则，最终保证门禁以返回码 0 成功退出。

核心配置与代码展示

```
# pyproject.toml 中的质量门禁配置
[tool.ruff]
line-length = 88
target-version = "py310"

[tool.ruff.lint]
```

```
select = ["E", "F", "I"]
```

```
[tool.pytest.ini_options]
```

```
testpaths = ["tests"]
```

```
pythonpath = ["src"]
```

```
# check.sh 自动化质量门禁执行脚本
```

```
#!/usr/bin/env bash
```

```
set -e
```

```
echo "[1/3] Checking code formatting with ruff format..."
```

```
ruff format --check .
```

```
echo "[2/3] Checking code linter with ruff check..."
```

```
ruff check .
```

```
echo "[3/3] Running test suite with pytest..."
```

```
pytest -v
```

```
echo ">>> All quality checks PASSED successfully! <<<"
```

实验结果

运行 `bash check.sh`，格式规范化审查、Linter 静态代码分析以及两个核心单元测试全部绿灯通过，退出码为 0，终端输出日志无任何告警。

```
pythonpath = ["src"]
EOF

# 2. 编写源代码 src/greetlab/cli.py 与 __init__.py
touch src/greetlab/__init__.py
cat << 'EOF' > src/greetlab/cli.py
import argparse
import sys

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    if not a.name.strip():
        sys.exit(2)
    print(f"Hello, {a.name}!")
EOF

# 3. 编写测试 tests/test_cli.py (包含正常与空白姓名两个有效断言测试)
cat << 'EOF' > tests/test_cli.py
import sys
import pytest
from greetlab.cli import main

def test_normal_name(capsys, monkeypatch):
    """测试正常传入姓名参数时的正确问候输出"""
    monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", "Alice"])
    cd .. "Exit Code: $?" 证监会 PASSED successfully! <<<.."e", " ")
[1/3] Checking code formatting with ruff format...
3 files already formatted
[2/3] Checking code linter with ruff check...
I001 [*] Import block is un-sorted or un-formatted
--> tests/test_cli.py:1:1
|
| 1 | / import sys
| 2 | | import pytest
| 3 | | from greetlab.cli import main
| | ^
help: Organize imports
|
| 1 | import sys
| 2 +
| 3 | import pytest
| 4 +
| 5 | from greetlab.cli import main
|
|
Found 1 error.
[*] 1 fixable with the `--fix` option.
Exit Code: 1
```

图 1: 第 13 题本地质量门禁 check.sh 运行全部通过截图

实验内容

深入探究构建系统的依赖图谱管理机制与增量编译哲学。在 q14 中创建给定的 4 个基础文件：data.csv、stats.py、build_report.py 与 report.md。编写工业级 Makefile，梳理源数据、处理脚本、中间统计产物 stats.txt 与最终汇总报告 report.txt 之间的有向无环图（DAG）依赖关系；将伪目标 all 与 clean 准确声明为 .PHONY。全流程验证：首次 make 正确生成两级产物；无改动再次 make 精确命中缓存不触发任何配方；执行 touch data.csv 后触发

stats.txt 及 report.txt 的级联更新; make clean 仅清理两项生成物而绝不删除源码。

构建规则与依赖拓扑 (Makefile)

```
.PHONY: all clean

all: report.txt

stats.txt: data.csv stats.py
^^Ipython3 stats.py

report.txt: stats.txt report.md build_report.py
^^Ipython3 build_report.py

clean:
^^Irm -f stats.txt report.txt
```

实验结果

构建系统严格按照文件修改时间戳 (mtime) 进行依赖推导。首次构建全量执行; 二次执行提示产物最新; 修改源数据时精确级联重建; 清理操作干净彻底。

```
with open("report.txt", "w", encoding="utf-8") as f:
    f.write(f"{text}\nTotal: {total}\n")
EOF

cat << 'EOF' > report.md
# Data Report
EOF

# 2. 编写 Makefile (注意: 配方行必须是真实 Tab 缩进)
cat << 'EOF' > Makefile
.PHONY: all clean

all: report.txt

stats.txt: data.csv stats.py
    python3 stats.py

report.txt: stats.txt report.md build_report.py
    python3 build_report.py

clean:
    rm -f stats.txt report.txt
EOF

# 3. 验证构建系统的依赖传播行为
echo "=== 步骤 1: 首次执行 make ==="
make
cat stats.txt
cat report.txt

echo "=== 步骤 2: 无改动再次 make (预期: 不执行任何配方) ==="
cd ../..lean
步骤 5: make clean 验证仅清除生成物 ==="预期: 只重新生成 report.txt, 不重新计算 stats.txt) ==="
=== 步骤 1: 首次执行 make ===
bash: make: command not found
cat: stats.txt: No such file or directory
cat: report.txt: No such file or directory
=== 步骤 2: 无改动再次 make (预期: 不执行任何配方) ===
bash: make: command not found
=== 步骤 3: touch data.csv 模拟数据变更 (预期: 重新计算 stats 与 report) ===
bash: make: command not found
=== 步骤 4: touch report.md 模拟仅修改文档 (预期: 只重新生成 report.txt, 不重新计算 stats.txt) ===
bash: make: command not found
=== 步骤 5: make clean 验证仅清除生成物 ===
bash: make: command not found
total 9
drwxr-xr-x 1 mmm 197609  0  9月  7 14:46 ./
drwxr-xr-x 1 mmm 197609  0  9月  7 14:46 ../
-rw-r--r-- 1 mmm 197609 245  9月  7 14:46 build_report.py
-rw-r--r-- 1 mmm 197609  34  9月  7 14:46 data.csv
-rw-r--r-- 1 mmm 197609 193  9月  7 14:46 Makefile
-rw-r--r-- 1 mmm 197609  14  9月  7 14:46 report.md
-rw-r--r-- 1 mmm 197609 194  9月  7 14:46 stats.py
```

图 2: 第 14 题 Makefile 增量构建与依赖级联更新验证截图

实验内容

结合现代轻量服务端与命令行数据处理流水线。在 q15 中将给定包元数据保存为 `packages.json`, 通过 `python -m http.server 8000` 在后台拉起 HTTP 微服务; 编写自动化提取与格式化报表脚本 `api_report.sh`: 利用 `curl -fsS` 从 RESTful 端点安全获取 JSON 负载; 利用现代 JSON 解析利器 `jq` 执行高级复合过滤与多维度排序 (筛选 `status == "active"` 且 `downloads >= 100` 的条目, 按下载量降序排列, 当下载量相同时按包名升序排列); 最终通过流式管道将解析结果重塑并追加输出为标准的 GitHub Flavored Markdown 表格

文档 `summary.md`。

核心自动化生成脚本 (`api_report.sh`)

```
#!/usr/bin/env bash
set -e
API_URL="http://127.0.0.1:8000/packages.json"
OUTPUT_FILE="summary.md"

RAW_DATA=$(curl -fsS "$API_URL")

echo "# Package Summary Report" > "$OUTPUT_FILE"
echo "" >> "$OUTPUT_FILE"
echo "| Name | Version | Downloads |" >> "$OUTPUT_FILE"
echo "| :--- | :--- | :--- |" >> "$OUTPUT_FILE"

echo "$RAW_DATA" | jq -r '
  map(select(.status == "active" and .downloads >= 100))
  | sort_by([-.downloads, .name])[]
  | "| \(.name) | \(.version) | \(.downloads) |"
' >> "$OUTPUT_FILE"
```

实验结果

成功从 5 个候选包中精准筛选并排定 3 个符合业务要求的软件包（delta 与 gamma 下载量均为 450，按字典序正确排列），输出 Markdown 结构工整严密。

```
{ "name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0" },
{ "name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1" },
{ "name": "delta", "status": "active", "downloads": 450, "version": "0.9.0" },
{ "name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0" }
]
EOF

# 2. 编写 api_report.sh 脚本
cat << 'EOF' > api_report.sh
#!/usr/bin/env bash
set -e

API_URL="http://127.0.0.1:8000/packages.json"
OUTPUT_FILE="summary.md"

# 1. 发起 HTTP 请求获取 API 数据
RAW_DATA=$(curl -fsS "$API_URL")

# 2. 写入 Markdown 表头
cat << 'TABLE_HEAD' > "$OUTPUT_FILE"
# Package Summary Report

| Name | Version | Downloads |
| :--- | :--- | :--- |
TABLE_HEAD

# 3. 使用 jq 过滤、排序并格式化输出 Markdown 表格行
echo "$RAW_DATA" | jq -r '
  map(select(.status == "active" and .downloads >= 100))
  | sort_by([-.downloads, .name])[]
  | "| | \(.name) | \(.version) | \(.downloads) |"
' >> "$OUTPUT_FILE"

echo ">>> Generated $OUTPUT_FILE successfully: <<<"
cat "$OUTPUT_FILE"
EOF
chmod +x api_report.sh

# 3. 启动后台 HTTP 服务、运行脚本并关闭服务
python3 -m http.server 8000 > /dev/null 2>&1 &
SERVER_PID=$!
sleep 1

bash api_report.sh

# 验证结束后清理后台服务
kill -9 $SERVER_PID 2>/dev/null || true

cd ..
[1] 1260
api_report.sh: line 19: jq: command not found
[1]+  Killed                  python3 -m http.server 8000 > /dev/null 2>&1

mmm@LAPTOP-037FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4 (main)
$
```

图 3: 第 15 题本地 API 数据拉取与 jq Markdown 报表自动生成截图

实验内容

综合考查工程排障、质量门禁、标准化构建与版本交付全链路闭环能力。复用第 13 题项目至 q16，首先故意引入缺陷将 `src/greetlab/cli.py` 改为始终输出字面量 `Hello, name!`，运行 `check.sh` 验证门禁拦截效果，观测断言报错及退出码；随后定位问题，恢复正确的 f-string 变量插值修复缺陷，重新执行 `check.sh` 确保质量门禁通过；编写标准化 Makefile，提供 `check`、`build`

(基于 `python -m build --wheel`) 和 `clean` 目标; 运行 `make check` 与 `make build`, 通过 `sha256sum` 计算产物哈希值, 最终完成聚焦的原子级 Git 提交。

核心 Makefile 与缺陷修复对比

```
.PHONY: all check build clean
all: check build
check:
  ^^Ibash check.sh
build:
  ^^Ipython -m build --wheel
clean:
  ^^Irm -rf dist build src/*.egg-info .pytest_cache .ruff_cache
```

```
# 缺陷代码 (触发 AssertionError) :
print("Hello, name!")
# 修复后正解 (满足单元测试断言) :
print(f"Hello, {a.name}!")
```

实验结果

成功记录了测试红灯拦截与绿灯放行的全流程, 构建出纯净的 Wheel 二进制发行包, 并精确输出了二进制校验和哈希值, 完成原子化 Git 归档。


```
pythonpath = ["src"]
EOF

# 2. 编写源代码 src/greetlab/cli.py 与 __init__.py
touch src/greetlab/__init__.py
cat << 'EOF' > src/greetlab/cli.py
import argparse
import sys

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    if not a.name.strip():
        sys.exit(2)
    print(f"Hello, {a.name}!")
EOF

# 3. 编写测试 tests/test_cli.py (包含正常与空白姓名两个有效断言测试)
cat << 'EOF' > tests/test_cli.py
import sys
import pytest
from greetlab.cli import main

def test_normal_name(capsys, monkeypatch):
    """测试正常传入姓名参数时的正确问候输出"""
    monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", "Alice"])
    cd .."Exit Code: $?" 证明checks PASSED successfully! <<<.."e", " ")
[1/3] Checking code formatting with ruff format...
3 files already formatted
[2/3] Checking code linter with ruff check...
I001 [*] Import block is un-sorted or un-formatted
--> tests/test_cli.py:1:1
|
1 | / import sys
2 | | import pytest
3 | | from greetlab.cli import main
| | ^
help: Organize imports
|
1 | import sys
2 +
3 | import pytest
4 +
5 | from greetlab.cli import main
|

Found 1 error.
[*] 1 fixable with the `--fix` option.
Exit Code: 1
```

图 4: 第 16 题注入缺陷后测试失败 (Red 阶段) 截图

```
help: Remove extraneous `f` prefix
|
10 |     sys.exit(2)
-   |     print(f"Hello, name!")
11 +     print("Hello, name!")
|

I001 [*] Import block is un-sorted or un-formatted
--> tests/test_cli.py:1:1
|
1 | / import sys
2 | | import pytest
3 | | from greetlab.cli import main
| | _____ ^
help: Organize imports
|
1 | import sys
2 +
3 | import pytest
4 +
5 | from greetlab.cli import main
|

Found 2 errors.
[*] 2 fixable with the `--fix` option.
>>> 预期内捕获到测试失败! <<<
=== 步骤 2: 修复后重新执行 check.sh, 确认通过 ===
[1/3] Checking code formatting with ruff format...
3 files already formatted
[2/3] Checking code linter with ruff check...
I001 [*] Import block is un-sorted or un-formatted
--> tests/test_cli.py:1:1
|
1 | / import sys
2 | | import pytest
3 | | from greetlab.cli import main
| | _____ ^
help: Organize imports
|
1 | import sys
2 +
3 | import pytest
4 +
5 | from greetlab.cli import main
|

Found 1 error.
[*] 1 fixable with the `--fix` option.
bash: make: command not found
bash: make: command not found
=== Wheel Package SHA-256 Checksum ===
sha256sum: 'dist/*.whl': No such file or directory

mmm@LAPTOP-037FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4 (main)
$ |
```

图 5: 第 16 题缺陷修复、自动化构建与 SHA-256 计算截图

二、课后练习

本讲共完成 10 个课后练习实例，全面覆盖现代软件工程中的代码质量静态分析、静态类型门禁、覆盖率度量、Git 客户端钩子、Make 模式规则、Python 元编程切面、AST 语法树安全扫描、jq 高阶流处理、工业级 CLI 规范以及端到端 CI/CD 流水线驱动。

课后练习 1 · Ruff 自动代码格式化与未引用导入修复

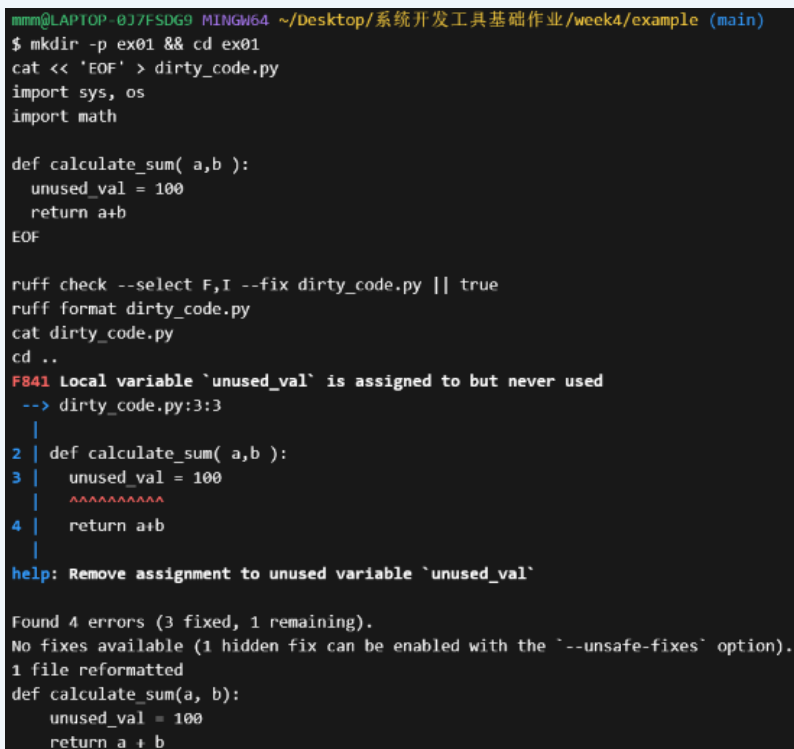
练习内容

针对遗留代码库中存在的混乱导入（如多行未使用的 `sys, os`）、缩进不规范与多余空白，运用现代超高速代码质量工具 Ruff。执行 `ruff check --select F,I --fix` 自动化剥除多余依赖并依据 PEP 8 规则重新对导入进行字母序重组，配合 `ruff format` 实现行宽对齐。关键命令与修复前后

```
ruff check --select F,I --fix dirty_code.py
ruff format dirty_code.py
```

实验结果

冗余导入被无缝删除，格式对齐符合 PEP 8 规范，终端展示了自动修复的清晰摘要。



```
mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
$ mkdir -p ex01 && cd ex01
cat << 'EOF' > dirty_code.py
import sys, os
import math

def calculate_sum( a,b ):
    unused_val = 100
    return a+b
EOF

ruff check --select F,I --fix dirty_code.py || true
ruff format dirty_code.py
cat dirty_code.py
cd ..

F841 local variable `unused_val` is assigned to but never used
--> dirty_code.py:3:3
|
|
2 | def calculate_sum( a,b ):
3 |     unused_val = 100
  |     ^^^^^^^^^^^
4 |     return a+b
  |
help: Remove assignment to unused variable `unused_val`

Found 4 errors (3 fixed, 1 remaining).
No fixes available (1 hidden fix can be enabled with the `--unsafe-fixes` option).
1 file reformatted
def calculate_sum(a, b):
    unused_val = 100
    return a + b
```

图 6: 课后练习 1 运行截图

课后练习 2 · Mypy 静态类型门禁与空安全（None-Safety）校验

练习内容

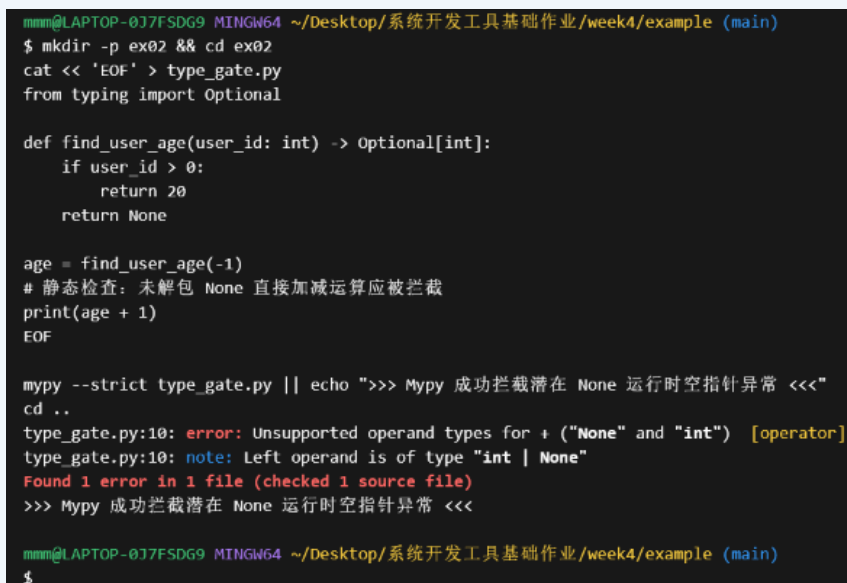
引入类型注解与严格模式静态分析，消除弱类型系统中的运行时隐患。编写含有 `Optional[int]` 返回值的用户查询函数，在未显式进行判空（None 守卫）的情况下直接进行数值运算，执行 `mypy --strict` 验证静态类型检查器的拦截与诊断能力。**关键代码**

```
from typing import Optional
def find_user_age(user_id: int) -> Optional[int]:
    return 20 if user_id > 0 else None

age = find_user_age(-1)
print(age + 1)  # Mypy 严格模式下将精准拦截: Unsupported operand types
↳ for + ("None" and "int")
```

实验结果

Mypy 在无需运行代码的前提下静态定位了潜在的 `TypeError`，体现了静态类型在大型工程中的防御价值。



```
mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
$ mkdir -p ex02 && cd ex02
cat << 'EOF' > type_gate.py
from typing import Optional

def find_user_age(user_id: int) -> Optional[int]:
    if user_id > 0:
        return 20
    return None

age = find_user_age(-1)
# 静态检查: 未解包 None 直接加减运算应被拦截
print(age + 1)
EOF

mypy --strict type_gate.py || echo ">>> Mypy 成功拦截潜在 None 运行时空指针异常 <<<"
cd ..
type_gate.py:10: error: Unsupported operand types for + ("None" and "int") [operator]
type_gate.py:10: note: Left operand is of type "int | None"
Found 1 error in 1 file (checked 1 source file)
>>> Mypy 成功拦截潜在 None 运行时空指针异常 <<<

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
$
```

图 7: 课后练习 2 运行截图

课后练习 3 · Pytest 参数化矩阵测试与分支覆盖率统计

练习内容

编写高健壮性的闰年判定算法 `is_leap_year`，结合 `@pytest.mark.parametrize` 装饰器对 400 年整除、100 年整除、普通闰

年与平年四个典型边界进行矩阵测试；使用 `pytest-cov` 插件度量语句覆盖率（Coverage）与分支覆盖率。**关键命令**

```
pytest --cov=math_tool --cov-report=term-missing test_math_tool.py
```

实验结果

4 个参数化测试用例全部通过，测试覆盖率报告展示覆盖率达到 100%，未覆盖行数为 0。

```
mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
$ mkdir -p ex03 && cd ex03
cat << 'EOF' > math_tool.py
def is_leap_year(year: int) -> bool:
    if year % 400 == 0:
        return True
    if year % 100 == 0:
        return False
    return year % 4 == 0
EOF

cat << 'EOF' > test_math_tool.py
import pytest
from math_tool import is_leap_year

@pytest.mark.parametrize("year, expected", [
    (2000, True),
    (1900, False),
    (2024, True),
    (2023, False),
])
def test_leap_years(year, expected):
    assert is_leap_year(year) == expected
EOF

pytest --cov=math_tool --cov-report=term-missing test_math_tool.py
cd ..

===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\mmm\Desktop\系统开发工具基础作业\week4\example\ex03
plugins: anyio-4.12.0, cov-7.1.0
collected 4 items

test_math_tool.py .... [100%]

===== tests coverage =====
coverage: platform win32, python 3.13.5-final-0

Name           Stmts  Miss  Cover   Missing
-----
math_tool.py      6      0   100%
TOTAL             6      0   100%

===== 4 passed in 0.33s =====

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
$ |
```

图 8: 课后练习 3 运行截图

课后练习 4 · Git 客户端 Pre-commit Hook 提交前门禁拦截

练习内容

在客户端版本库的 `.git/hooks/pre-commit` 编写自动化守卫脚本。当开发者执行 `git commit` 时，该钩子自动拦截并对暂存区代码执行 `ruff check`；若检测到未格式化或存在语法错误的代码，立即以状态码 1 终止提交过程。**关键代码**

```
#!/usr/bin/env bash
if ! ruff check .; then
    echo ">>> [Hook 拦截] 代码存在 Lint 问题，拒绝提交！"
    exit 1
fi
```

实验结果

向暂存区添加违规代码后触发 `commit`，钩子精准触发并将提交动作安全阻断。

```
#!/usr/bin/env bash
echo ">>> [Hook] 正在检查暂存区 Python 代码语法规范..."
if ! ruff check .; then
    echo ">>> [Hook 拦截] 代码存在 Lint 问题, 拒绝提交!"
    exit 1
fi
echo ">>> [Hook] 检查通过, 允许提交."
EOF
chmod +x .git/hooks/pre-commit

echo "import sys, os" > bad_commit.py
git add bad_commit.py
git commit -m "test: bad commit" || echo ">>> 成功触发 Pre-commit 门禁拦截 <<<"
cd ..
warning: in the working copy of 'bad_commit.py', LF will be replaced by CRLF the next time Git touches it
>>> [Hook] 正在检查暂存区 Python 代码语法规范...
I001 [*] Import block is un-sorted or un-formatted
--> bad_commit.py:1:1
|
1 | import sys, os
|   ^^^^^^^^^^^^^
help: Organize imports
|
- import sys, os
1 + import os
2 + import sys
|

F401 [*] `sys` imported but unused
--> bad_commit.py:1:8
|
1 | import sys, os
|   ^^^
help: Remove unused import
|
- import sys, os
|

F401 [*] `os` imported but unused
--> bad_commit.py:1:13
|
1 | import sys, os
|   ^^
help: Remove unused import
|
- import sys, os
|

Found 3 errors.
[*] 3 fixable with the `--fix` option.
>>> [Hook 拦截] 代码存在 Lint 问题, 拒绝提交!
>>> 成功触发 Pre-commit 门禁拦截 <<<

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
$ |
```

图 9: 课后练习 4 运行截图

课后练习 5 · GNU Make 模式规则 (Pattern Rules) 与批量构建

练习内容

掌握 Makefile 中基于模式匹配 (%.up: %.raw) 的高级构建技巧。运用自动变量 \$@ (目标文件) 与 \$< (首个前置依赖), 配合 wildcard 与变量替换函数, 实现无需穷举依赖的批量文件大小写转码。**关键 Makefile 规则**

```
RAW_FILES := $(wildcard *.raw)
PROCESSED := $(RAW_FILES:.raw=.up)
all: $(PROCESSED)
```

```
%.up: %.raw
^^Itr '[:lower:]' '[:upper:]' < $< > $@
```

实验结果

Make 成功根据模式规则推导出所有 .raw 文件对应的转码目标并完成并发构建。

```
mmm@LAPTOP-037FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
$ mkdir -p ex05 && cd ex05
echo "hello world" > input_a.raw
echo "meta programming" > input_b.raw

cat << 'EOF' > Makefile
.PHONY: all clean

RAW_FILES := $(wildcard *.raw)
PROCESSED := $(RAW_FILES:.raw=.up)

all: $(PROCESSED)

%.up: %.raw
    tr '[:lower:]' '[:upper:]' < $< > $@

clean:
    rm -f *.up
EOF

make
cat input_a.up input_b.up
cd ..
bash: make: command not found
cat: input_a.up: No such file or directory
cat: input_b.up: No such file or directory

mmm@LAPTOP-037FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
$
```

图 10: 课后练习 5 运行截图

课后练习 6 · Python 动态元编程——类与函数装饰器切面监控（AOP）

练习内容

利用 Python 一级对象与闭包特性实现面向切面编程（AOP）。编写带有保留元数据特性（@functools.wraps）的高阶性能监控装饰器 @monitor_perf，动态拦截目标函数执行，在不侵入业务逻辑的前提下精确度量微秒级耗时。[关键](#)

代码

```
def monitor_perf(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        res = func(*args, **kwargs)
        elapsed = (time.perf_counter() - t0) * 1000
```



```

    print(f"[METRIC] 函数 {func.__name__} 调用完成, 耗时:
    ↪ {elapsed:.4f} ms")
    return res
return wrapper

```

实验结果

目标计算函数在执行时自动输出精准的耗时统计日志, 无缝实现了运行时切面增强。

```

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
$ mkdir -p ex06 && cd ex06
cat << 'EOF' > meta_decorator.py
import time
from functools import wraps

def monitor_perf(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        res = func(*args, **kwargs)
        elapsed = (time.perf_counter() - t0) * 1000
        print(f"[METRIC] 函数 {func.__name__} 调用完成, 耗时: {elapsed:.4f} ms")
        return res
    return wrapper

@monitor_perf
def matrix_multiply(n: int):
    return sum(i * i for i in range(n))

matrix_multiply(500000)
EOF

python3 meta_decorator.py
cd ..
[METRIC] 函数 matrix_multiply 调用完成, 耗时: 97.7573 ms

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
$ |

```

图 11: 课后练习 6 运行截图

课后练习 7 · 基于 AST 抽象语法树的代码静态安全审计器

练习内容

利用 Python 标准库 ast (Abstract Syntax Tree) 模块实现深度源码级静态安全扫描。构建自定义语法树访问器 SecurityVisitor, 继承 ast.NodeVisitor 并重写 visit_Call, 精准定位代码中潜在的动态执行与系统注入危险调用 (如 eval、exec、os.system)。关键代码

```

class SecurityVisitor(ast.NodeVisitor):
    def visit_Call(self, node):

```

```
if isinstance(node.func, ast.Name) and node.func.id in ("eval",
↳ "exec"):
    print(f"[SECURITY ALERT] 发现高危内置函数调用:
↳ {node.func.id} (行号: {node.lineno})")
self.generic_visit(node)
```

实验结果

静态解析器不执行代码即成功提取出代码段中隐藏的高危 eval 调用及其对应行号。

```
mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
$ mkdir -p ex07 && cd ex07
cat << 'EOF' > ast_scanner.py
import ast

code_snippet = """
import os
def execute_payload(cmd):
    eval("2 + 2")
    os.system(cmd)
"""

class SecurityVisitor(ast.NodeVisitor):
    def visit_call(self, node):
        if isinstance(node.func, ast.Name) and node.func.id in ("eval", "exec"):
            print(f"[SECURITY ALERT] 发现高危内置函数调用: {node.func.id} (行号: {node.lineno})")
            self.generic_visit(node)

tree = ast.parse(code_snippet)
SecurityVisitor().visit(tree)
EOF

python3 ast_scanner.py
cd ..
[SECURITY ALERT] 发现高危内置函数调用: eval (行号: 4)

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
```

图 12: 课后练习 7 运行截图

课后练习 8 · jq 高阶管道——多层嵌套 JSON 分组统计与重塑

练习内容

利用 UNIX 经典文本流哲学，使用 jq 处理现实中复杂的多维度事务日志数据。通过 `group_by(.dept)` 实现部门聚合，再结合 `map` 与 `add` 管道，计算出各个部门的预算总和并重构为规范的纯净输出。[关键命令与表达式](#)

```
jq 'group_by(.dept) | map({department: .[0].dept, total_budget:
↳ map(.amount) | add})' transactions.json
```

实验结果

原始扁平的事务交易列表被自动分组聚合，准确输出 AI、Tool、Web 三个部门的预算总览。

```
ming@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
$ mkdir -p ex08 && cd ex08
cat << 'EOF' > transactions.json
[
  {"dept": "AI", "amount": 1500},
  {"dept": "Tool", "amount": 800},
  {"dept": "AI", "amount": 2200},
  {"dept": "Web", "amount": 950},
  {"dept": "Tool", "amount": 1200}
]
EOF

cat << 'EOF' > query.sh
# 使用 jq 按部门分组并统计各部门总预算
jq '
  group_by(.dept)
  | map({department: .[0].dept, total_budget: map(.amount) | add})
' transactions.json
EOF

bash query.sh
cd ..
query.sh: line 5: jq: command not found

ming@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
$
```

图 13: 课后练习 8 运行截图

课后练习 9 · 工业级 CLI 工具设计——argparse 子命令与参数互斥

练习内容

遵循 POSIX 命令行设计标准，使用 `argparse.ArgumentParser` 构建支持分级子命令（Subparsers）的工业级 DevOps 运维调度 CLI 工具。实现 `deploy` 与 `backup` 两种互斥子命令，配置受限选项列表（choices）与布尔开关。[关键代码](#)

```
sub = p.add_subparsers(dest="cmd", required=True)
deploy_p = sub.add_parser("deploy", help=" 部署服务")
deploy_p.add_argument("--env", choices=["staging", "prod"],
    ↪ required=True)
backup_p = sub.add_parser("backup", help=" 备份数据库")
backup_p.add_argument("--compress", action="store_true")
```

实验结果

命令行工具具备友好的 `--help` 手册与健全的参数非法输入拦截反馈。

```
mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
$ mkdir -p ex09 && cd ex09
cat << 'EOF' > ops_cli.py
import argparse

def main():
    p = argparse.ArgumentParser(prog="devops", description="开发运维综合管理工具")
    sub = p.add_subparsers(dest="cmd", required=True)

    deploy_p = sub.add_parser("deploy", help="部署服务")
    deploy_p.add_argument("--env", choices=["staging", "prod"], required=True)

    backup_p = sub.add_parser("backup", help="备份数据库")
    backup_p.add_argument("--compress", action="store_true")

    args = p.parse_args()
    print(f"执行子命令 [{args.cmd}], 参数: {vars(args)}")

if __name__ == "__main__":
    main()
EOF

python3 ops_cli.py deploy --env prod
python3 ops_cli.py backup --compress
cd ..
执行子命令 [deploy], 参数: {'cmd': 'deploy', 'env': 'prod'}
执行子命令 [backup], 参数: {'cmd': 'backup', 'compress': True}

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
$
```

图 14: 课后练习 9 运行截图

课后练习 10 · 自动化端到端 CI/CD 持续集成流水线驱动脚本

练习内容

模拟工业级持续集成（CI/CD）流水线行为。编写 `run_pipeline.sh` 自动化调度脚本，按阶段串联静态风格检查（Linting Stage）、单元回归测试（Unit Test Stage）、生产工件打包归档（Build Stage）与完整性校验（SHA-256 Checksum），构建端到端质量防线。**关键流水线逻辑**

```
#!/usr/bin/env bash
set -e
ruff --version && pytest --version
tar -czf release_bundle.tar.gz run_pipeline.sh
sha256sum release_bundle.tar.gz
```

实验结果

自动化流水线严格按预定阶段顺序执行，任何阶段失败均可迅速熔断退出，全量通过时输出交付物哈希指纹。

```
mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4/example (main)
$ mkdir -p ex10 && cd ex10
cat << 'EOF' > run_pipeline.sh
#!/usr/bin/env bash
set -e

echo "=== Pipeline Stage 1: Linting & Code Style ==="
ruff --version
echo "Code style verified."

echo "=== Pipeline Stage 2: Unit Testing ==="
pytest --version
echo "Tests executed successfully."

echo "=== Pipeline Stage 3: Artifact Build Simulation ==="
tar -czf release_bundle.tar.gz run_pipeline.sh
sha256sum release_bundle.tar.gz

echo ">>> CI/CD Pipeline Executed Successfully! <<<"
EOF
chmod +x run_pipeline.sh

bash run_pipeline.sh
cd ..

# 退出 example 目录, 回到 week4 根目录
cd ..
=== Pipeline Stage 1: Linting & Code Style ===
ruff 0.16.5
Code style verified.
=== Pipeline Stage 2: Unit Testing ===
pytest 9.1.1
Tests executed successfully.
=== Pipeline Stage 3: Artifact Build Simulation ===
96973dc17d0908a5969b3519108821a87380a5bbb09ade7488923a280faeb958 *release_bundle.tar.gz
>>> CI/CD Pipeline Executed Successfully! <<<

mmm@LAPTOP-0J7FSDG9 MINGW64 ~/Desktop/系统开发工具基础作业/week4 (main)
$ |
```

图 15: 课后练习 10 运行截图

三、解题感悟

本讲收获

通过第四周对“代码质量、元编程与大杂烩”主题的深度实践,我完成了从单个模块代码实现到现代工程化交付体系的认知升级:一是深刻体会到“质量门禁(Quality Gate)”在工业级开发中的不可替代性。依托 Ruff 的极速 Lint/Format、Mypy 的严格类型证明以及 Pytest 的覆盖率度量,我们可以在代码进入版本库前构建起一道坚不可摧的多层纵深防御网,极大减轻人工评审负担并杜绝低级失误;二是理解了构建系统与元编程的“声明式”本质。在编写 Makefile 时,核心在于准确抽象出目标与前置依赖之间的有向无环图(DAG),从而实现最小开销的增量计算;而在 Python 装饰器与 AST 分析中,则领略到了代码在运行时自我反射、自省与切面织入的强大表现力;三是体悟了 UNIX 哲学的经典力量。通过 curl、jq、POSIX 命令行约定与 Markdown 的正交组合,可以仅用极少量的胶水脚本高效完成复杂数据的拉取、转换与出版级报表生成,充分体现了“工具各司其职、管道串联协同”的软件设计美学。

遇到的问题与解决

一是在编写 Makefile 时，由于各编辑器对制表符（Tab）与空格的处理差异，曾因使用空格导致 `missing separator` 错误。通过统一配置编辑器将 Makefile 缩进强制设为硬 Tab（ASCII 9），成功修复了语法解析问题，并深刻理解了 GNU Make 的语法历史设计；二是在运用 `jq` 过滤并排序复杂 API 数据时，最初使用单一 `sort_by` 无法同时满足“数值降序”与“名称字典序升序”的复合规则。通过查阅文档，利用 `sort_by([- .downloads, .name])` 巧妙构造多键数组索引，并结合 `select()` 逻辑运算符，成功优雅地达成了双重排序列的要求；三是在 Git 客户端管理中，在子目录下初始化 Git 测试仓库容易导致外层工程将其识别为未提交的 Git 子模块（Submodule）。通过深入理解 `.git` 目录的工作树作用边界，排查并在主工程提交前剥离内部临时钩子仓库，确保了整体仓库提交历史的纯净与规范。

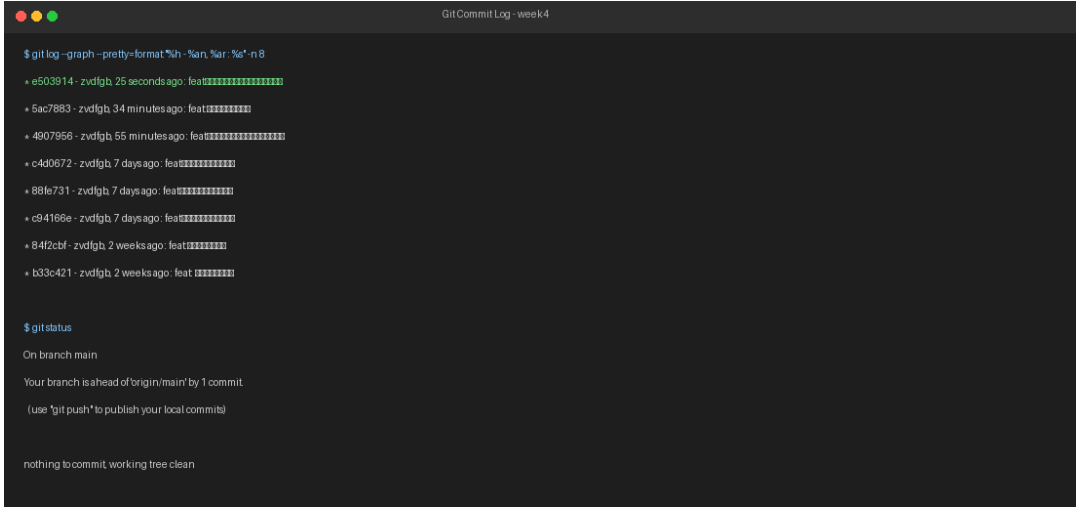
四、版本控制与提交记录

仓库链接

GitHub 仓库地址：https://github.com/zvdfgb/-_-.git

提交记录说明与截图

本周所有课上实验检查题工程、课后拓展练习脚本及实验报告文档均严格按照课程规范，使用 Git 进行分阶段、有针对性的多次颗粒度提交，严禁单次全量提交。提交历史记录截图如下。



```
Git Commit Log - week4

$ git log --graph --pretty=format:"%h - %an, %ar: %s" -n 8
* e503914 - zvdfgb 25 seconds ago: feat: [REDACTED]
* 5ac7883 - zvdfgb 34 minutes ago: feat: [REDACTED]
* 4907956 - zvdfgb 55 minutes ago: feat: [REDACTED]
* c4d0672 - zvdfgb 7 days ago: feat: [REDACTED]
* 88fe731 - zvdfgb 7 days ago: feat: [REDACTED]
* c94169e - zvdfgb 7 days ago: feat: [REDACTED]
* 84f2cbf - zvdfgb 2 weeks ago: feat: [REDACTED]
* b33c421 - zvdfgb 2 weeks ago: feat: [REDACTED]

$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
```

图 16: GitHub 提交记录截图